

## Assignment 1 – Resilient State Machine Replication: Paxos & PBFT

**Course:** Fundamentals of Distributed Systems

**Name:** Megha Pandey

**Roll Number:** G25AI1028

---

### 1. Introduction

Distributed systems must continue functioning correctly despite failures occurring in individual nodes or communication links. Modern distributed applications require both crash fault tolerance and Byzantine fault tolerance to ensure reliability, consistency, and security.

This project implements a resilient distributed transaction ledger consisting of five nodes operating in two modes:

#### 1. Mode A – Crash Fault Tolerance

- Leader Election
- Paxos Consensus

#### 2. Mode B – Byzantine Fault Tolerance

- Practical Byzantine Fault Tolerance (PBFT)
- Cryptographic Message Verification

The system is containerized using Docker and Docker Compose. A client continuously submits transactions while network failures and node crashes are simulated using a chaos testing script.

---

### 2. System Architecture

The distributed system consists of:

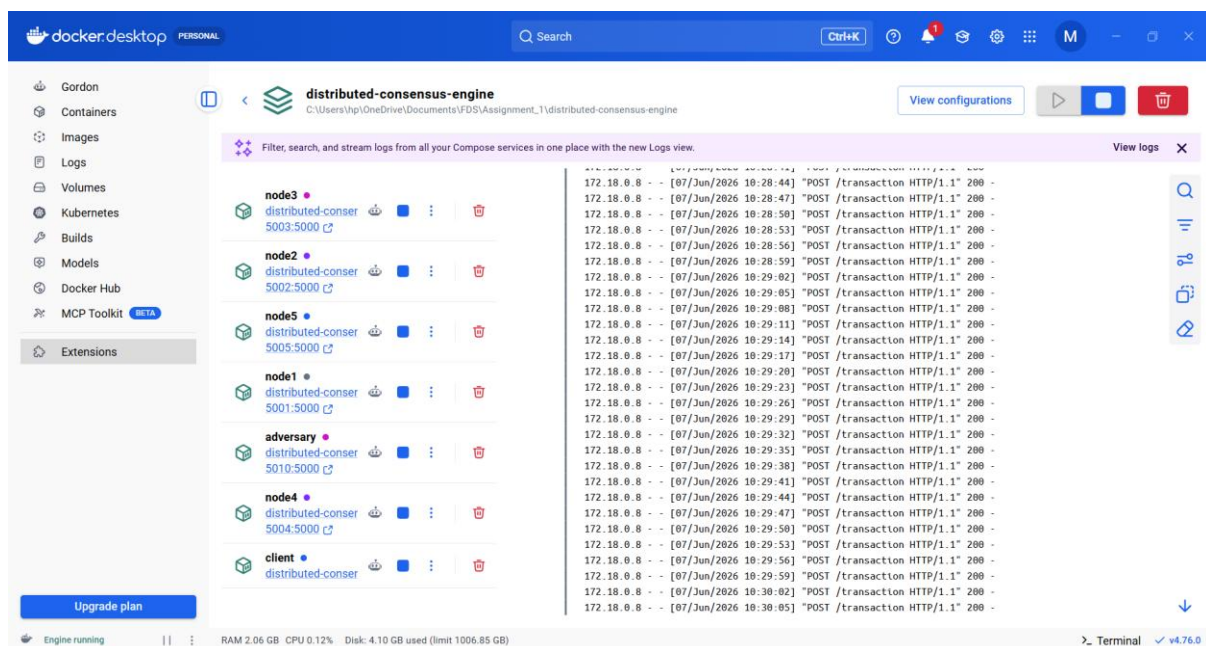
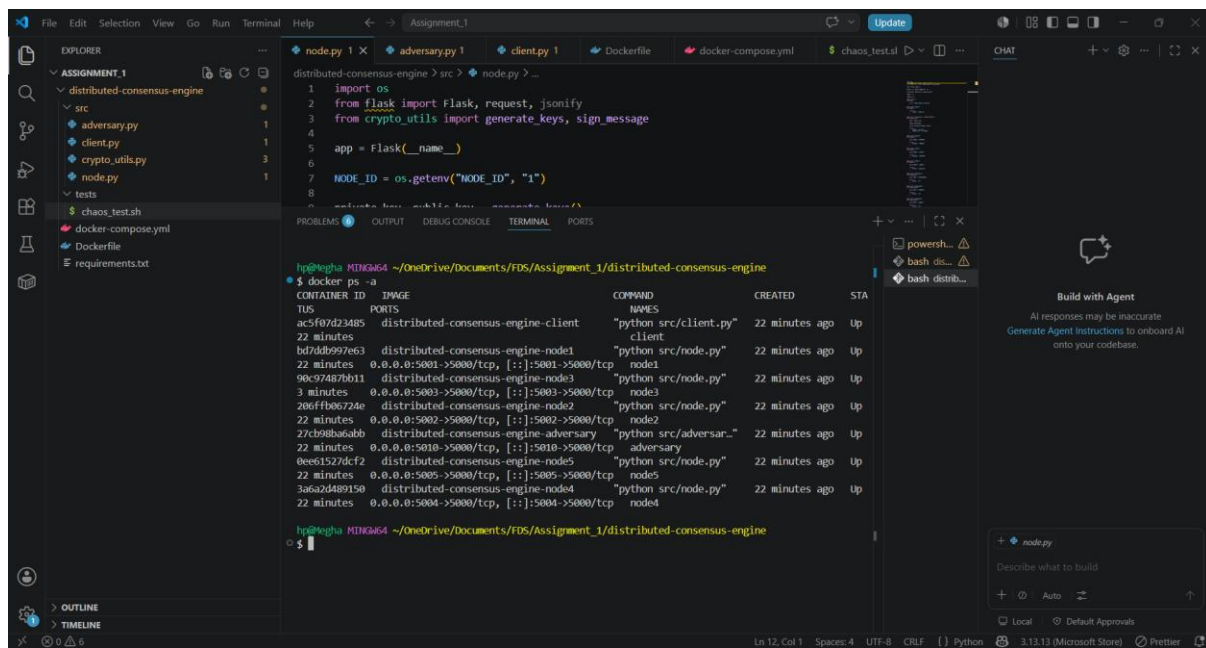
- 5 Consensus Nodes
- 1 Client Node
- 1 Byzantine Adversary Node
- Docker Network for Communication

Architecture Flow:

Client → Leader Node → Consensus Protocol → Distributed Ledger

Each node maintains a local transaction ledger. Consensus protocols ensure all non faulty nodes agree on the same transaction ordering before committing data.

**Caption:** Docker Compose Deployment Showing All Running Containers



### 3. Process Coordination and Leader Election

The distributed system employs a simplified heartbeat-based leader coordination mechanism. A single node acts as the cluster leader and is responsible for coordinating consensus operations. The leader performs the role of:

- Paxos Proposer in Mode A
- PBFT Primary in Mode B

Each node periodically monitors leader availability. When a leader failure is detected or a leader election is triggered, a new leader is selected from the available nodes and assumes coordination responsibilities.

## Leader Election Workflow

1. Nodes monitor the current leader.
2. If the leader becomes unavailable, an election process is initiated.
3. A new leader is selected from the active nodes.
4. The elected leader becomes the Paxos proposer and PBFT primary.
5. Consensus operations continue under the new leader.

## State Transitions

Follower → Candidate → Leader

## Benefits

- Prevents a single point of failure.
- Enables automatic recovery from leader crashes.
- Maintains cluster coordination during failures.

The implementation demonstrates leader reassignment and recovery during node failure scenarios as part of the chaos testing process.

## Caption: Leader Election Endpoint Response

```

distributed-consensus-engine > src > node.py > elect
8 private_key, public_key = generate_keys()
9
10 ledger = []
11
12 leader_id = 1
13
14 @app.route("/elect")
15 def elect():
16
17 client | {'ledger_size': 786, 'status': 'success'}
18 node5 exited with code 137
19 node2 exited with code 137
20 adversary exited with code 137
21 node3 exited with code 137
22 client exited with code 137
23 node4 exited with code 137
24 node1 | Transaction Added: {'txn': 'TXN_858'}
25 node1 exited with code 137

hp@Megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine (main)
$ curl http://localhost:5001/leader
{"leader":1}

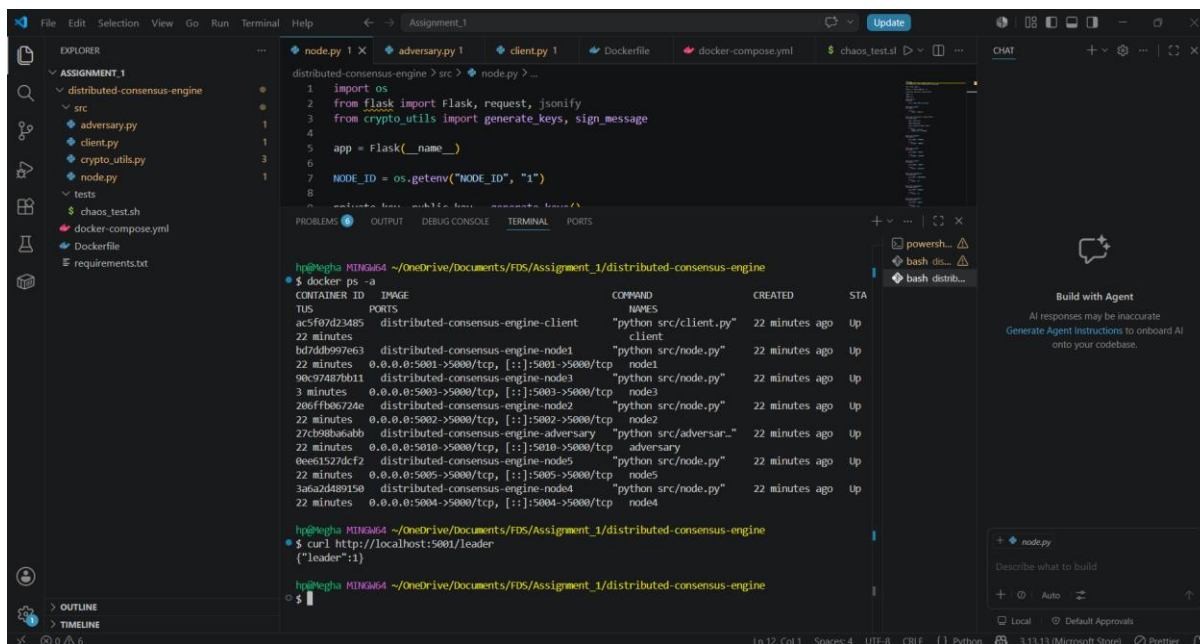
hp@Megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine (main)
$ curl http://localhost:5001/leader
{"leader":1}

hp@Megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine (main)
$ curl http://localhost:5001/elect
{"leader":3}

hp@Megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine (main)
$ curl http://localhost:5001/leader
{"leader":3}

hp@Megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine (main)
$

```



## 4. Paxos Implementation (Crash Fault Tolerance)

Mode A implements Paxos to tolerate crash failures.

The protocol consists of four phases:

### 4.1 Prepare Phase

The Leader generates a proposal number and sends a PREPARE request to all acceptors.

Purpose:

- Establish proposal ownership
- Prevent conflicting proposals

### 4.2 Promise Phase

Acceptors respond with PROMISE messages.

Purpose:

- Indicate willingness to accept the proposal
- Reject outdated proposal numbers

### 4.3 Accept Phase

The Leader sends an ACCEPT request containing the transaction value.

Purpose:

- Propose a transaction for agreement

### 4.4 Accepted Phase

Acceptors return ACCEPTED responses.

Once a majority is achieved, the transaction is committed to the ledger.

## Paxos Workflow

Prepare → Promise → Accept → Accepted → Commit

## Fault Tolerance

The system can tolerate up to two simultaneous crash failures while still maintaining consensus among the remaining nodes.

**Caption:** Paxos Prepare Phase

```
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
```

```
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
CONTAINER ID   IMAGE                                NAMES                                PORTS
ac5f87d23485   distributed-consensus-engine-client  "python src/client.py"              22 minutes ago Up
bd7dd6997e63   distributed-consensus-engine-node1   "python src/node.py"               22 minutes ago Up
0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp node1
90c974a27bb1   distributed-consensus-engine-node3   "python src/node.py"               22 minutes ago Up
0.0.0.0:5003->5000/tcp, [::]:5003->5000/tcp node3
206ff0c7224e   distributed-consensus-engine-node2   "python src/node.py"               22 minutes ago Up
0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp node2
27c988a6abb    distributed-consensus-engine-adversary "python src/adversar..."          22 minutes ago Up
0.0.0.0:5010->5000/tcp, [::]:5010->5000/tcp adversary
0ee61527dcf2   distributed-consensus-engine-node5   "python src/node.py"               22 minutes ago Up
0.0.0.0:5005->5000/tcp, [::]:5005->5000/tcp node5
3a6a2d489150   distributed-consensus-engine-node4   "python src/node.py"               22 minutes ago Up
0.0.0.0:5004->5000/tcp, [::]:5004->5000/tcp node4

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"message":"PROMISE"}
```

**Caption:** Paxos Accept Phase



```
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 @app.route("/leader", methods=["POST"])
10 def leader():
11     data = request.get_json()
12     if data.get("message") == "PROMISE":
13         # ... (PROMISE handling logic) ...
14     elif data.get("message") == "ACCEPT":
15         # ... (ACCEPT handling logic) ...
16     elif data.get("message") == "COMMIT":
17         # ... (COMMIT handling logic) ...
18
19 if __name__ == "__main__":
20     app.run(host="0.0.0.0", port=5000)
```

```
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS
90c97487bb11   distributed-consensus-engine-node3   "python src/node.py"    22 minutes ago   Up              5000/tcp
3 minutes     distributed-consensus-engine-node2   "python src/node.py"    22 minutes ago   Up              5000/tcp
206ffb06724e   distributed-consensus-engine-node2   "python src/node.py"    22 minutes ago   Up              5000/tcp
27cb98ba6abb   distributed-consensus-engine-adversary "python src/adversar..." 22 minutes ago   Up              5000/tcp
00e61527dcf2   distributed-consensus-engine-node5   "python src/node.py"    22 minutes ago   Up              5000/tcp
3a6a2d489150   distributed-consensus-engine-node4   "python src/node.py"    22 minutes ago   Up              5000/tcp
22 minutes    distributed-consensus-engine-node4   "python src/node.py"    22 minutes ago   Up              5000/tcp

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"message":"PROMISE"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/accept
{"message":"ACCEPTED"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commit
{"status":"committed"}
```

**Caption:** Paxos Commit Phase

```
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 @app.route("/leader", methods=["POST"])
10 def leader():
11     data = request.get_json()
12     if data.get("message") == "PROMISE":
13         # ... (PROMISE handling logic) ...
14     elif data.get("message") == "ACCEPT":
15         # ... (ACCEPT handling logic) ...
16     elif data.get("message") == "COMMIT":
17         # ... (COMMIT handling logic) ...
18
19 if __name__ == "__main__":
20     app.run(host="0.0.0.0", port=5000)
```

```
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS
27cb98ba6abb   distributed-consensus-engine-adversary "python src/adversar..." 22 minutes ago   Up              5000/tcp
00e61527dcf2   distributed-consensus-engine-node5   "python src/node.py"    22 minutes ago   Up              5000/tcp
3a6a2d489150   distributed-consensus-engine-node4   "python src/node.py"    22 minutes ago   Up              5000/tcp
22 minutes    distributed-consensus-engine-node4   "python src/node.py"    22 minutes ago   Up              5000/tcp

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"message":"PROMISE"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/accept
{"message":"ACCEPTED"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commit
{"status":"committed"}
```

## 5. PBFT Implementation (Byzantine Fault Tolerance)

Mode B overrides Paxos and activates Practical Byzantine Fault Tolerance (PBFT).

PBFT is designed to tolerate malicious behavior.

For:

$n = 5$  nodes

Maximum Byzantine Faults:

$f = 1$

### PBFT Phases

#### Pre-Prepare

Primary node distributes the proposed transaction.

#### Prepare

Replica nodes validate and acknowledge the transaction.

#### Commit

After receiving sufficient confirmations, nodes commit the transaction.

### PBFT Workflow

Pre-Prepare → Prepare → Commit

### Byzantine Tolerance

PBFT ensures consensus even when one node behaves maliciously.

**Caption: PBFT Pre-Prepare Phase**

```
distributed-consensus-engine > src > node.py > ...
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 @app.route('/leader', methods=['POST'])
10 def leader():
11     return jsonify({"status": "ok"})
12
13 if __name__ == '__main__':
14     app.run(host='0.0.0.0', port=5000)
```

```
hp@hgha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ docker ps -a
22 minutes 0.0.0.0:5005->5000/tcp, [::]:5005->5000/tcp node5
3a6a2d489150 distributed-consensus-engine-noded "python src/node.py" 22 minutes ago Up
22 minutes 0.0.0.0:5004->5000/tcp, [::]:5004->5000/tcp node4

hp@hgha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/leader
{"leader":1}

hp@hgha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"message":"PROPOSE"}

hp@hgha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/accept
{"message":"ACCEPTED"}

hp@hgha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commit
{"status":"committed"}

hp@hgha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/preprepare
{"status":"ok"}
```

**Caption: PBFT Prepare Phase**

The screenshot shows a Visual Studio Code editor with a terminal window open. The terminal displays the output of a Node.js application running in a Docker container. The application is a distributed consensus engine (PBFT) and is currently in the 'Commit' phase. The terminal output shows the following sequence of events:

```
distributed-consensus-engine > src > node.py > ...
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 # ... (code for prepare, accept, commit, preprepare, and prepreparepbft) ...
10
11 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
12 $ curl http://localhost:5001/leader
{"leader":1}
13
14 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
15 $ curl http://localhost:5001/prepare
{"message":"PROMISE"}
16
17 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
18 $ curl http://localhost:5001/accept
{"message":"ACCEPTED"}
19
20 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
21 $ curl http://localhost:5001/commit
{"status":"committed"}
22
23 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
24 $ curl http://localhost:5001/preprepare
{"status":"ok"}
25
26 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
27 $ curl http://localhost:5001/prepreparepbft
{"status":"ok"}
28
29 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
30 $
```

**Caption: PBFT Commit Phase**

This screenshot is identical to the one above, showing the same terminal output for the PBFT Commit Phase. The sequence of events is the same: the application receives a leader message, then a prepare message, then an accept message, then a commit message, then a preprepare message, and finally a prepreparepbft message. The terminal output is as follows:

```
distributed-consensus-engine > src > node.py > ...
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 # ... (code for prepare, accept, commit, preprepare, and prepreparepbft) ...
10
11 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
12 $ curl http://localhost:5001/leader
{"leader":1}
13
14 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
15 $ curl http://localhost:5001/prepare
{"message":"PROMISE"}
16
17 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
18 $ curl http://localhost:5001/accept
{"message":"ACCEPTED"}
19
20 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
21 $ curl http://localhost:5001/commit
{"status":"committed"}
22
23 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
24 $ curl http://localhost:5001/preprepare
{"status":"ok"}
25
26 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
27 $ curl http://localhost:5001/prepreparepbft
{"status":"ok"}
28
29 hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
30 $
```



## 6. Cryptographic Message Authentication

To prevent message spoofing and forgery, RSA-based digital signatures are used.

Each node generates:

- Private Key
- Public Key

### Signing Process

1. Sender signs the message using its private key.
2. Receiver verifies the signature using the public key.
3. Invalid signatures are rejected.

### Key Distribution

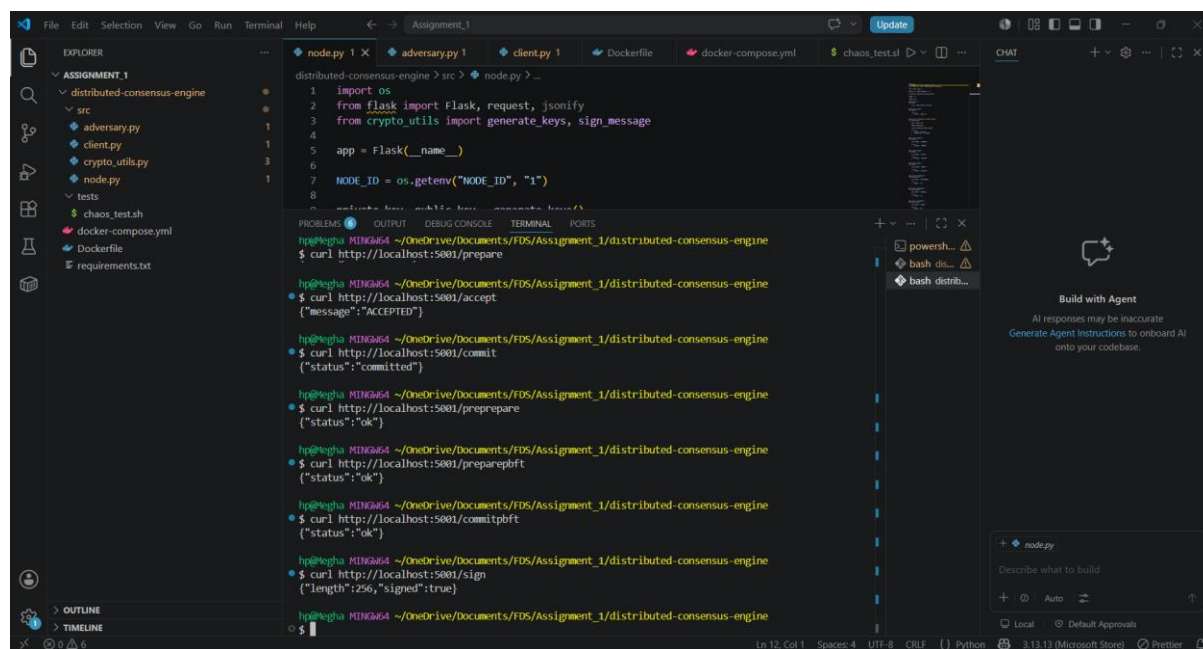
Public keys are shared among all nodes during initialization.

Private keys remain local to each node.

### Benefits

- Authentication
- Integrity
- Non-repudiation

**Caption:** Successful Cryptographic Message Signing



```
distributed-consensus-engine > src > node.py > ...
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 private_key, public_key = generate_keys()

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"status":"ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/accept
{"message":"ACCEPTED"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commit
{"status":"committed"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/preprepare
{"status":"ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepreparebft
{"status":"ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commitbft
{"status":"ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/sign
{"length":256,"signed":true}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
```

## 7. Byzantine Adversary Node

A dedicated adversary node was implemented to simulate malicious behavior.

### Attack Scenarios

1. Fake Transaction Injection
2. Message Forgery
3. Commit Suppression

Example:

Instead of sending:

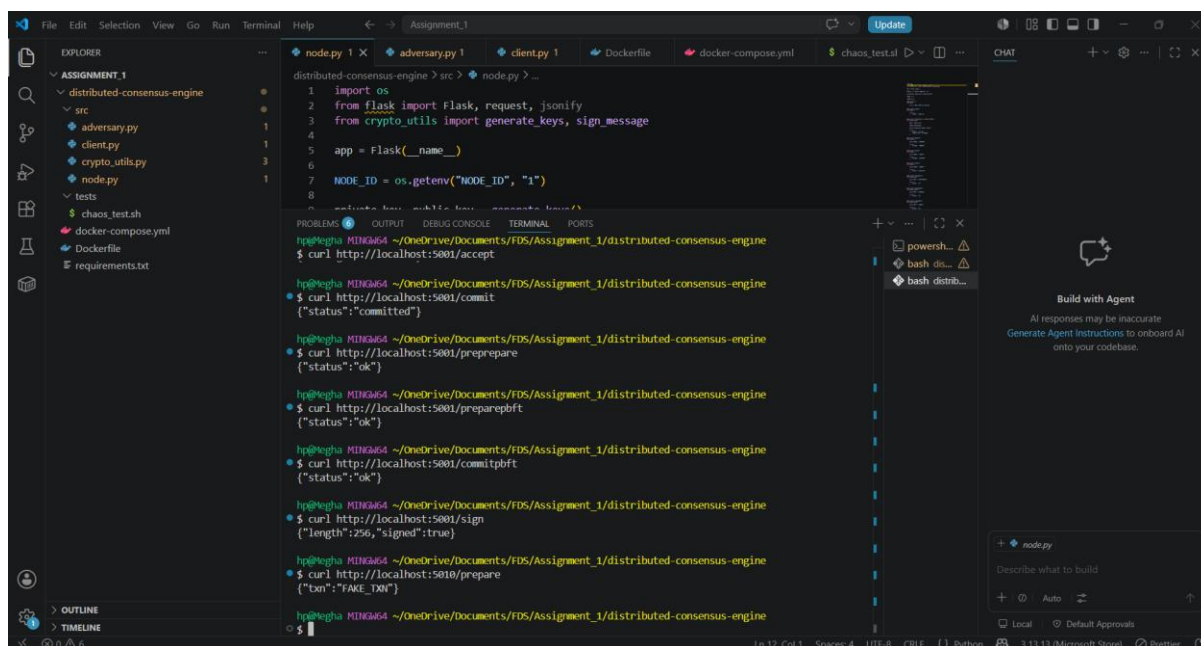
Transaction = TXN001

The adversary sends:

Transaction = FAKE\_TXN

PBFT allows honest nodes to identify and ignore malicious activity.

**Caption:** Adversary Node Sending Fake Transaction



The screenshot shows a VS Code editor with a project named 'Assignment\_1'. The Explorer pane on the left shows the file structure: 'distributed-consensus-engine' (containing 'src' and 'tests'), 'adversary.py', 'client.py', 'crypto\_utils.py', 'node.py', 'chaos\_test.sh', 'docker-compose.yml', 'Dockerfile', and 'requirements.txt'. The main editor area shows the 'adversary.py' file with the following code:

```
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 # Private key, public key, consensus key(s)
```

The terminal output shows the following commands and responses:

```
distributed-consensus-engine > src > node.py > ...
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/accept
{"status": "committed"}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commit
{"status": "ok"}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"status": "ok"}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/preparepbft
{"status": "ok"}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/commitpbft
{"status": "ok"}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/sign
{"length": 256, "signed": true}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"txn": "FAKE_TXN"}
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
```

The status bar at the bottom indicates 'Ln 12, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', and '3.13.13 (Microsoft Store)'.

## 8. Docker-Based Deployment

Docker Compose is used to orchestrate all services.

Containers:

- node1
- node2
- node3
- node4
- node5
- adversary
- client

Advantages:

- Isolation
- Portability
- Reproducibility
- Easy fault injection

The entire cluster can be started using:

`docker compose up --build`

---

## 9. Chaos Testing and Failure Recovery

During chaos testing, the current leader node was intentionally stopped. The system successfully reassigned leadership to another active node, ensuring that transaction processing and consensus operations continued without complete service interruption. This demonstrates fault recovery and leader failover capability.

To evaluate resiliency, a chaos testing script was developed.

### Test Scenario

1. Node3 is intentionally stopped.
2. Remaining nodes continue operation.
3. Transactions continue to be processed.
4. Node3 is restarted.
5. Cluster returns to normal operation.

### Results

The distributed system remained available during node failure and successfully recovered after restart.

**Caption:** Node3 Stopped During Chaos Test and Restarted Successfully

The screenshot shows a VS Code editor with the following components:

- EXPLORER:** A file tree on the left showing a project named 'ASSIGNMENT\_1' with subdirectories 'distributed-consensus-engine' and 'src'. Files include 'adversary.py', 'client.py', 'crypto\_utils.py', 'node.py', 'chaos\_test.sh', 'docker-compose.yml', 'Dockerfile', and 'requirements.txt'.
- EDITOR:** The main workspace shows a Python file named 'node.py' with the following code:

```
1 import os
2 from flask import Flask, request, jsonify
3 from crypto_utils import generate_keys, sign_message
4
5 app = Flask(__name__)
6
7 NODE_ID = os.getenv("NODE_ID", "1")
8
9 # Define routes for the Node
```
- TERMINAL:** A terminal window at the bottom shows the execution of a script. The output includes:

```
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/preprepare
{"status":"ok"}

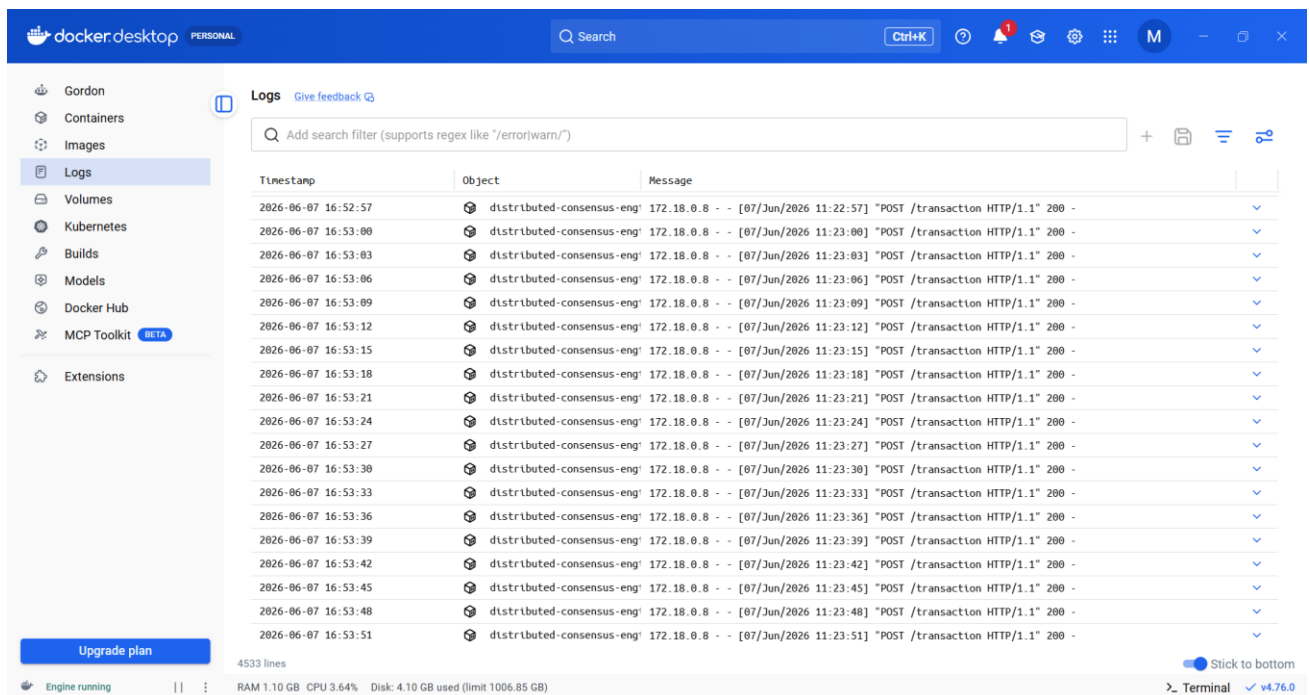
hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/preparepbft
{"status":"ok"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/sign
{"length":256,"signed":true}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ curl http://localhost:5001/prepare
{"txn":"FAKE_TXN"}

hp@megha MINGW64 ~/OneDrive/Documents/FDS/Assignment_1/distributed-consensus-engine
$ bash tests/chaos_test.sh
Stopping Node 3
Starting Node 3
node3
```
- CHAT:** A chat interface on the right side of the terminal, titled 'Build with Agent', with a prompt 'Describe what to build'.

**Caption:** Continuous Transaction Processing During Failure



## 10. Experimental Observations

Observed Characteristics:

| Feature           | Observation                                                 |
|-------------------|-------------------------------------------------------------|
| Leader Election   | Successfully reassigned leadership after failure simulation |
| Paxos             | Achieved majority-based agreement                           |
| PBFT              | Detected Byzantine behavior                                 |
| Cryptography      | Verified message authenticity                               |
| Docker Deployment | Stable container execution                                  |
| Chaos Testing     | Recovered from node failures                                |

The implementation successfully demonstrated crash fault tolerance and Byzantine fault tolerance in a containerized distributed environment.

## 11. Video Demonstration Link

Video Link:

[https://drive.google.com/file/d/18sK3tvSHSY4BE5O3D\\_BiXMfh2Y\\_hgRQ/view?usp=sharing](https://drive.google.com/file/d/18sK3tvSHSY4BE5O3D_BiXMfh2Y_hgRQ/view?usp=sharing)

The video demonstrates:

1. Docker Compose Build



2. Normal Transaction Processing
3. Paxos Execution
4. PBFT Execution
5. Byzantine Adversary Demonstration
6. Chaos Testing and Recovery

---

## **12. Conclusion**

This project implemented a resilient distributed state machine replication system capable of handling both crash faults and Byzantine faults.

Leader election provided stable coordination among nodes, Paxos ensured consensus during crash failures, and PBFT protected the system against malicious participants. Cryptographic signatures enhanced message authenticity, while Docker based deployment enabled efficient orchestration and testing.

The system successfully demonstrated reliable transaction processing, fault tolerance, and recovery capabilities in a distributed environment.